PL-SQL Lab 1

RESHMITHA K R

AM.SC.U4AIE23145

1. Do all the questions discussed in the class

a)

```
create or replace function sum(a int, b int, out c int)
as $$
begin
    c := a + b;
end;
$$ language plpgsql;
select sum(210, 430);
```

	sum integer
1	640

b)

```
create or replace function eo(n INT) returns TEXT as
$$
begin
    if n % 2 = 0 then
        return 'Even';
    else
        return 'Odd';
    end if;
end;
$$ language plpgsql;

select eo(27);
```

	eo text
1	Odd

c)

```
create or replace function sal(p_eno varchar) returns int as $$
declare
    emp_salary int;
begin
    select basic_sal into emp_salary
    from emp
    where eno = p_eno;

    return emp_salary;
end;
$$ language plpgsql;

select sal('E1')
```

	sal integer
1	10000

d)

```
create or replace function emp_c(p_deptno varchar) returns int as $$
declare
    emp_count int;
begin
    select count(*) into emp_count
    from emp
    where dept_no = p_deptno;

    return emp_count;
end;
$$ language plpgsql;

select emp_c('D1')
```

	emp_c integer
1	2

e)

```
create or replace function sal_check(p_eno emp.eno%type) returns void as $$
declare
    emp_dept emp.dept_no%type;
    emp_salary emp.basic_sal%type;
    dept_avg_salary emp.basic_sal%type;
begin
    select dept_no, basic_sal into emp_dept, emp_salary
    from emp
    where eno = p_eno;

    select avg(basic_sal) into dept_avg_salary
    from emp
    where dept_no = emp_dept;

    if dept_avg_salary > emp_salary then
        update emp
        set basic_sal = basic_sal * 1.10
        where eno = p_eno;
    end if;
end;
$$ language plpgsql;

select sal_check('E1')
```

	eno [PK] character varying (2)	ename character varying (10)	basic_sal integer	incentive integer	dept_no character varying (2)	mgr_id character varying (2)	joiningdate date
1	E2	jkl	15000	4000	D1	E2	2022-04-07
2	E3	mno	13000	3000	D2	E3	2023-05-02
3	E1	ghi	11000	2000	D1	E1	2024-09-07

	eno [PK] character varying (2)	ename character varying (10)	basic_sal integer	incentive integer	dept_no character varying (2)	mgr_id character varying (2)	joiningdate date
1	E2	jkl	15000	4000	D1	E2	2022-04-07
2	E3	mno	13000	3000	D2	E3	2023-05-02
3	E1	ghi	12100	2000	D1	E1	2024-09-07

f)

```
create or replace function dname(p_ename emp.ename%type) returns text as $$
declare
    dept_name dept.dname%type;
begin
    select d.dname into dept_name
    from emp e
    join dept d on e.dept_no = d.dno
    where e.ename = p_ename;

    return dept_name;
end;
$$ language plpgsql;

select dname('jkl')
```

	dname text
1	abc

2. Consider the following schema . Emp(eno, ename, sal)
EMP_PROJ(eno, pno, prjt_hrs).

a) Write a function ProjectLoad that returns the total project working hours for the given eno.

```
create or replace function projectload(p_eno emp.eno%type
returns numeric as $$
declare
    v_total_hrs emp_proj.prjt_hrs%type;
begin
    select coalesce(sum(prjt_hrs), 0)
    into v_total_hrs
    from emp_proj
    where eno = p_eno;

    return v_total_hrs;
end;
$$ language plpgsql;

select projectload('e1')
```

	projectload numeric
1	25

b) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between currrent sal and average salary .

```
create or replace function bavg()
returns text as $$
declare
    avgsal emp.sal%type;
begin
    select avg(sal)
    into avgsal
    from emp;

    update emp
    set sal = sal + (avgsal - sal)
    where sal < avgsal;

    return 'salary updated for employees below average';
end;
$$ language plpgsql;
select updatesalbelowavg();
select * from emp;
```

	eno [PK] character varying (2)	ename character varying (20)	sal integer
1	e2	jkl	15000
2	e1	abc	13889
3	e3	mno	13889

3. Consider the following tables
   Item(ino, iname, unit_price)
   Transaction(tr_no,ino,qty)

   Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```
create or replace function checkdeleteitem(p_ino item.ino%type)
returns text as $$
declare
    v_count integer;
begin
    select count(*)
    into v_count
    from transaction
    where ino = p_ino;

    if v_count < 2 then
        delete from item
        where ino = p_ino;

        return 'item deleted';
    else
        return 'item not deleted';
    end if;
end;
$$ language plpgsql;

select checkdeleteitem('i1');
```

	checkdeleteitem text
1	item not deleted

4. Consider a Bank database which includes the following tables.
   ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
   BRANCHES(br_no, br_name, loc)
   CUSTOMER(cno, cname, c_type)

   a) Write a function that accepts a threshold value and a customer number . The program updates the c_type based on the threshold value. Ie. If balance> threshold then class A, else class B.

```
create or replace function updatecusttype(p_threshold numeric, p_cust_no customer.cno%type)
returns text as $$
declare
    v_total_bal numeric;
    v_new_type char(1);
begin
    select coalesce(sum(bal),0)
    into v_total_bal
    from accounts
    where cust_no = p_cust_no;

    if v_total_bal > p_threshold then
        v_new_type := 'a';
    else
        v_new_type := 'b';
    end if;

    update customer
    set c_type = v_new_type
    where cno = p_cust_no;

    return 'customer type updated';
end;
$$ language plpgsql;

select updatecusttype(20000, 'c1');
select * from customer where cno = 'c1';
```

	cno [PK] character varying (5)	cname character varying (20)	c_type character (1)
1	c1	arun	a

b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.

```
create or replace function closebranch(p_old_br branches.br_no%type,
                                       p_new_br branches.br_no%type)
returns text as $$
begin
    update accounts
    set br_no = p_new_br
    where br_no = p_old_br;

    delete from branches
    where br_no = p_old_br;

    return 'branch closed and accounts transferred';
end;
$$ language plpgsql;

select closebranch('b1', 'b2');
select * from accounts;
select * from branches;
```

	ac_no [PK] character varying (5)	br_no character varying (5)	cust_no character varying (5)	ac_type character varying (10)	bal numeric
1	a3	b2	c2	savings	30000
2	a4	b3	c3	savings	12000
3	a5	b3	c3	current	5000
4	a6	b2	c4	savings	40000
5	a1	b2	c1	savings	15000
6	a2	b2	c1	current	8000

	br_no [PK] character varying (5)	br_name character varying (20)	loc character varying (20)
1	b2	lake_branch	lakeside
2	b3	mall_branch	megacity

c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
create or replace function safewithdraw(p_ac_no accounts.ac_no%type,
                                        p_amount numeric)
returns text as $$
declare
    v_balance numeric;
begin
    select bal
    into v_balance
    from accounts
    where ac_no = p_ac_no
    for update;

    if v_balance >= p_amount then
        update accounts
        set bal = bal - p_amount
        where ac_no = p_ac_no;

        return 'withdrawal successful';
    else
        return 'insufficient funds';
    end if;
end;
$$ language plpgsql;

select safewithdraw('a1', 5000);
select * from accounts where ac_no = 'a1';
```

	ac_no [PK] character varying (5)	br_no character varying (5)	cust_no character varying (5)	ac_type character varying (10)	bal numeric
1	a1	b2	c1	savings	10000